

DSA Tutorial 2: Time Complexity and Big O Notation

This chapter covers Time Complexity and Big O Notation in a clear and beginner friendly way. After completing this tutorial, students will understand how to measure the efficiency of their code and what Big O Notation means.

1. Introduction

In the previous tutorial, we learned that DSA helps us write faster and more efficient code. But how do we measure how fast our code is?

We cannot just use a stopwatch — because the same code runs at different speeds on different computers. A program running on a powerful laptop will always be faster than on an old computer, even if the code is bad.

We need a way to measure efficiency that is independent of the computer. This is where **Time Complexity** and **Big O Notation** come in.

2. What is Time Complexity

Time Complexity measures how the running time of an algorithm grows as the size of the input grows.

The key word here is **grows**. We do not care about the exact time in seconds. We care about the pattern — what happens when our data gets bigger?

For example:

- If we have 10 students, finding a name takes 10 steps.
- If we have 100 students, finding a name takes 100 steps.
- If we have 1 million students, finding a name takes 1 million steps.

The time grows proportionally with the input size. This is a specific type of time complexity.

3. What is Big O Notation

Big O Notation is the standard way to express time complexity. It describes the worst case scenario — the maximum number of steps an algorithm might take.

We write it as **O(something)** — where the something describes the growth pattern.

The letter **n** represents the size of the input — the number of elements we are working with.

4. $O(1)$ — Constant Time

$O(1)$ means the algorithm always takes the same amount of time — no matter how big the input is.

```
numbers = [10, 20, 30, 40, 50]

first = numbers[0]
print(first)
```

No matter if the list has 5 elements or 5 million elements — accessing the first element always takes exactly 1 step.

Real life example: Opening a specific page in a book when you already know the page number. It does not matter if the book has 100 pages or 1000 pages — you go directly to that page.

$O(1)$ is the best possible time complexity.

5. $O(n)$ — Linear Time

$O(n)$ means the time grows linearly with the input size. If input doubles, time doubles.

```
numbers = [10, 20, 30, 40, 50]

target = 40
for num in numbers:
    if num == target:
        print("Found!")
```

In the worst case, we check every element. If there are n elements, we do n comparisons.

- 10 elements = 10 steps
- 100 elements = 100 steps
- 1 million elements = 1 million steps

Real life example: Finding a specific person by checking every seat one by one in a cinema hall.

6. $O(n^2)$ — Quadratic Time

$O(n^2)$ means the time grows with the square of the input size. This happens when we have a loop inside a loop.

```
numbers = [3, 1, 4, 1, 5]
```

```
for i in range(len(numbers)):
    for j in range(len(numbers)):
        print(numbers[i], numbers[j])
```

For every element, we go through all elements again.

- 10 elements = 100 steps
- 100 elements = 10,000 steps
- 1000 elements = 1,000,000 steps

Real life example: Every person in a room shakes hands with every other person. As more people join, the total handshakes grow very fast.

$O(n^2)$ is considered slow for large inputs. We try to avoid it when possible.

7. $O(\log n)$ — Logarithmic Time

$O(\log n)$ means the time grows very slowly even as the input grows very fast. This is the second best time complexity after $O(1)$.

```
# Binary search example
numbers = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]

target = 13
low = 0
high = len(numbers) - 1

while low <= high:
    mid = (low + high) // 2
    if numbers[mid] == target:
        print("Found at index", mid)
        break
    elif numbers[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
```

Each step cuts the problem in half.

- 8 elements = 3 steps
- 1024 elements = 10 steps
- 1 million elements = only 20 steps

Real life example: Finding a word in a dictionary. You open the middle, see if your word comes before or after, then open the middle of that half. You never check every word.

8. $O(n \log n)$ — Linearithmic Time

$O(n \log n)$ is slightly slower than $O(n)$ but much better than $O(n^2)$. Most efficient sorting algorithms have this complexity.

Examples: Merge Sort, Quick Sort.

- 10 elements = ~33 steps
 - 1000 elements = ~10,000 steps
 - 1 million elements = ~20 million steps (compare to n^2 which would be 1 trillion)
-

9. Comparing All Complexities

Here is how all complexities compare for different input sizes.

Input Size (n)	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
10	1	3	10	33	100
100	1	7	100	664	10,000
1,000	1	10	1,000	9,966	1,000,000
1,000,000	1	20	1,000,000	20,000,000	1,000,000,000,000

From best to worst: $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$

10. How to Calculate Time Complexity

When looking at code, follow these simple rules.

Rule 1 — Single loop = $O(n)$

```
for i in range(n):  
    print(i)
```

Rule 2 — Two nested loops = $O(n^2)$

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Rule 3 — Drop constants

```
for i in range(n):  
    print(i)  
  
for i in range(n):  
    print(i)
```

This is $O(2n)$ — but we drop the constant 2 and write it as $O(n)$.

Rule 4 — Drop smaller terms

If complexity is $O(n^2 + n)$, we drop the smaller term n and write $O(n^2)$. The larger term dominates.

Rule 5 — Halving each step = $O(\log n)$

If the input is cut in half each step, the complexity is $O(\log n)$.

11. Practical Example — Comparing Two Solutions

Problem: Find if a number exists in a list.

Solution 1 — Linear Search $O(n)$

```
def linear_search(arr, target):  
    for item in arr:  
        if item == target:  
            return True  
    return False
```

Solution 2 — Using a Set $O(1)$

```
def set_search(arr, target):  
    lookup = set(arr)  
    return target in lookup
```

For 1 million elements:

- Linear Search = up to 1 million checks.
- Set Search = 1 check always.

This is why choosing the right data structure matters so much.

12. Conclusion of Tutorial 2

In this tutorial, students have learned:

1. What Time Complexity is and why we measure it.
2. What Big O Notation is and how it works.
3. $O(1)$ — Constant time with real examples.
4. $O(n)$ — Linear time with real examples.
5. $O(n^2)$ — Quadratic time and why to avoid it.
6. $O(\log n)$ — Logarithmic time and why it is so powerful.
7. $O(n \log n)$ — Used in efficient sorting algorithms.
8. How to compare all complexities.
9. Simple rules to calculate time complexity from code.
10. A practical example showing why complexity matters.

In the next tutorial, we will learn about Space Complexity — measuring memory usage of algorithms.

This completes DSA Tutorial 2.